ComicCraft — AI Comic Story Creator

Complete Source Code & Execution Guide

Project basis: the ComicCraft starter implementation created for this conversation. The uploaded project URL points to the public GitHub repository; the repository README states the Enchanted Forest demo prompt and the Uvicorn command **python -m uvicorn app:app --reload**. This PDF contains the complete source code available in the generated project folder.

Execution

```
1. Open the ComicCraft folder in VS Code.
2. Create a virtual environment:
   python -m venv venv
3. Activate it on Windows:
   venv\Scripts\activate
4. Install dependencies:
   pip install -r requirements.txt
5. Copy .env.example to .env and add GEMINI_API_KEY.
6. Run the generated Flask version:
   python app.py
7. Open:
   http://127.0.0.1:5000

Note: the linked repository README uses:
   python -m uvicorn app:app --reload
That command applies to the linked repository's ASGI app; the generated project in this PDF is the Flask implementation supplied in this convers
```

app.py

```
import os, json, re, base64, sqlite3, uuid
from datetime import datetime
from functools import wraps
from io import BytesIO

from flask import Flask, render_template, request, jsonify, redirect, url_for, session, send_file, flash
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename
from dotenv import load_dotenv
from google import genai
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
from reportlab.lib.utils import ImageReader

load_dotenv()
app = Flask(__name__)
app.secret_key = os.getenv("FLASK_SECRET_KEY", "change-this-secret-key")
app.config["MAX_CONTENT_LENGTH"] = 2 * 1024 * 1024
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "comiccraft.db")
IMAGE_DIR = os.path.join(BASE_DIR, "static", "generated")
os.makedirs(IMAGE_DIR, exist_ok=True)

TEXT_MODEL = os.getenv("GEMINI_TEXT_MODEL", "gemini-3.8-flash")
IMAGE_MODEL = os.getenv("GEMINI_IMAGE_MODEL", "gemini-3.1-flash-image")

def get_client():
    key = os.getenv("GEMINI_API_KEY")
    if not key:
        raise RuntimeError("GEMINI_API_KEY is missing. Add it to your .env file.")
    return genai.Client(api_key=key)

def db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    with db() as conn:
        conn.execute("""CREATE TABLE IF NOT EXISTS users(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            email TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL
        )""")
        conn.execute("""CREATE TABLE IF NOT EXISTS comics(
            id TEXT PRIMARY KEY,
            user_id INTEGER NOT NULL,
            title TEXT NOT NULL,
            genre TEXT,
            story_json TEXT NOT NULL,
            created_at TEXT NOT NULL,
            FOREIGN KEY(user_id) REFERENCES users(id)
        )""")

init_db()

def login_required(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        if "user_id" not in session:
            return jsonify({"error": "Please log in first."}), 401
        return fn(*args, **kwargs)
    return wrapper

@app.route("/")
def index():
    return render_template("index.html", username=session.get("username"))

@app.route("/register", methods=["POST"])
def register():
    data = request.get_json() or {}
    username = (data.get("username") or "").strip()
    email = (data.get("email") or "").strip().lower()
    password = data.get("password") or ""
    if not username or not email or len(password) < 6:
        return jsonify({"error": "Enter a username, valid email and password (at least 6 characters)."}), 400
    try:
        with db() as conn:
            cur = conn.execute(
                "INSERT INTO users(username,email,password) VALUES(?,?,?)",
                (username, email, generate_password_hash(password))
            )
            session["user_id"] = cur.lastrowid
            session["username"] = username
        return jsonify({"message": "Account created successfully."})
    except sqlite3.IntegrityError:
        return jsonify({"error": "Username or email already exists."}), 409

@app.route("/login", methods=["POST"])
def login():
```

```
    data = request.get_json() or {}
    with db() as conn:
        user = conn.execute("SELECT * FROM users WHERE email=?", ((data.get("email") or "").lower(),)).fetchone()
    if not user or not check_password_hash(user["password"], data.get("password") or ""):
        return jsonify({"error": "Incorrect email or password."}), 401
    session["user_id"], session["username"] = user["id"], user["username"]
    return jsonify({"message": "Welcome back!"})

@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("index"))

def extract_json(text):
    text = (text or "").strip()
    text = re.sub(r"^```(?:json)?\s*|\s*```$", "", text, flags=re.I)
    start, end = text.find("{"), text.rfind("}")
    if start < 0 or end < start:
        raise ValueError("Gemini did not return valid JSON. Please try again.")
    return json.loads(text[start:end+1])

@app.route("/api/generate-story", methods=["POST"])
@login_required
def generate_story():
    data = request.get_json() or {}
    idea = (data.get("idea") or "").strip()
    genre = (data.get("genre") or "Adventure").strip()
    style = (data.get("style") or "Colorful modern comic book").strip()
    count = max(3, min(int(data.get("panels", 4)), 8))
    if len(idea) < 8:
        return jsonify({"error": "Please enter a story idea with at least 8 characters."}), 400

    prompt = f"""
You are a professional comic-book writer. Create an original {genre} comic from this idea:
{idea}

Return ONLY valid JSON with this exact structure:
{{
 "title": "short engaging title",
 "summary": "2 sentence summary",
 "characters": [
   {{"name":"character name","description":"specific visual appearance, clothing, age range and personality"}}
 ],
 "panels": [
   {{"number":1,"scene":"visual scene description","dialogue":"short speech bubble text","caption":"optional narration"}}
 ]
}}
Create exactly {count} panels. Keep character appearance consistent in every scene.
Dialogue must be short enough for a comic speech bubble. No markdown fences.
"""
    try:
        response = get_client().models.generate_content(model=TEXT_MODEL, contents=prompt)
        story = extract_json(response.text)
        story["genre"] = genre
        story["style"] = style
        story["panels"] = story.get("panels", [])[:count]
        if not story["panels"]:
            raise ValueError("No comic panels were returned.")
        return jsonify({"story": story})
    except Exception as e:
        app.logger.exception("Story generation failed")
        return jsonify({"error": str(e)}), 500

@app.route("/api/generate-panel", methods=["POST"])
@login_required
def generate_panel():
    data = request.get_json() or {}
    panel = data.get("panel") or {}
    story = data.get("story") or {}
    title = story.get("title", "Comic")
    style = story.get("style", "Colorful modern comic book")
    character_text = "; ".join(
        f'{c.get("name","Character")}: {c.get("description","")}'
        for c in story.get("characters", [])
    )
    prompt = f"""
Create ONE finished comic-book illustration for the following scene.
Comic title: {title}
Art style: {style}. Use polished professional sequential-art illustration,
clear composition, expressive faces, strong readable silhouettes, rich colors,
cinematic lighting, and a clean comic-panel border.
Character reference (keep designs consistent): {character_text}
Scene: {panel.get("scene","A dramatic comic scene")}
Do not add lettering, words, captions, logos, watermarks, or speech bubbles.
Leave some uncluttered space where dialogue can be placed later.
"""
    try:
        interaction = get_client().interactions.create(model=IMAGE_MODEL, input=prompt)
        image_data = getattr(getattr(interaction, "output_image", None), "data", None)
        if not image_data:
            # SDK versions may expose image data inside model_output steps.
            for step in getattr(interaction, "steps", []) or []:
                if getattr(step, "type", "") == "model_output":
                    for block in getattr(step, "content", []) or []:
```

```
                        if getattr(block, "type", "") == "image":
                            image_data = getattr(block, "data", None)
                            break
                if image_data:
                    break
        if not image_data:
            raise RuntimeError("The image model returned no image. Check model access and API quota.")
        raw = base64.b64decode(image_data)
        filename = f"{uuid.uuid4().hex}.png"
        with open(os.path.join(IMAGE_DIR, filename), "wb") as f:
            f.write(raw)
        return jsonify({"image_url": url_for("static", filename=f"generated/{filename}")})
    except Exception as e:
        app.logger.exception("Panel image generation failed")
        return jsonify({"error": str(e)}), 500

@app.route("/api/save-comic", methods=["POST"])
@login_required
def save_comic():
    data = request.get_json() or {}
    story = data.get("story")
    if not story or not story.get("title") or not story.get("panels"):
        return jsonify({"error": "Generate a comic before saving."}), 400
    comic_id = uuid.uuid4().hex
    with db() as conn:
        conn.execute(
            "INSERT INTO comics(id,user_id,title,genre,story_json,created_at) VALUES(?,?,?,?,?,?)",
            (comic_id, session["user_id"], story["title"], story.get("genre",""), json.dumps(story),
             datetime.utcnow().isoformat(timespec="seconds"))
        )
    return jsonify({"message": "Comic saved.", "id": comic_id})

@app.route("/api/my-comics")
@login_required
def my_comics():
    with db() as conn:
        rows = conn.execute(
            "SELECT id,title,genre,created_at FROM comics WHERE user_id=? ORDER BY created_at DESC",
            (session["user_id"],)
        ).fetchall()
    return jsonify({"comics": [dict(r) for r in rows]})

@app.route("/api/comic/<comic_id>")
@login_required
def get_comic(comic_id):
    with db() as conn:
        row = conn.execute("SELECT * FROM comics WHERE id=? AND user_id=?",
                           (comic_id, session["user_id"])).fetchone()
    if not row:
        return jsonify({"error": "Comic not found."}), 404
    return jsonify({"story": json.loads(row["story_json"])})

@app.route("/api/comic/<comic_id>/pdf")
@login_required
def download_pdf(comic_id):
    with db() as conn:
        row = conn.execute("SELECT * FROM comics WHERE id=? AND user_id=?",
                           (comic_id, session["user_id"])).fetchone()
    if not row:
        return "Comic not found", 404
    story = json.loads(row["story_json"])
    output = BytesIO()
    pdf = canvas.Canvas(output, pagesize=A4)
    page_w, page_h = A4
    pdf.setTitle(story.get("title", "ComicCraft"))
    for i, panel in enumerate(story.get("panels", [])):
        if i:
            pdf.showPage()
        pdf.setFont("Helvetica-Bold", 20)
        pdf.drawString(42, page_h - 48, story.get("title", "ComicCraft")[:55])
        pdf.setFont("Helvetica", 10)
        pdf.drawString(42, page_h - 67, f'{story.get("genre","Comic")}  |  Panel {i+1}')
        image_url = panel.get("image_url", "")
        image_path = os.path.join(BASE_DIR, "static", image_url.replace("/static/", "")) if image_url.startswith("/static/") else ""
        if image_path and os.path.isfile(image_path):
            try:
                pdf.drawImage(ImageReader(image_path), 42, 175, width=page_w-84, height=page_h-285,
                              preserveAspectRatio=True, anchor="c", mask="auto")
            except Exception:
                pass
        pdf.setFont("Helvetica-Bold", 13)
        pdf.drawString(42, 135, f'Panel {i+1}')
        pdf.setFont("Helvetica", 11)
        dialogue = panel.get("dialogue", "")
        caption = panel.get("caption", "")
        y = 115
        for line in [dialogue, caption]:
            if line:
                words, current = line.split(), ""
                for word in words:
                    if len(current + " " + word) > 88:
                        pdf.drawString(42, y, current)
                        y -= 15
                        current = word
```

```
                    else:
                        current = (current + " " + word).strip()
                if current:
                    pdf.drawString(42, y, current)
                    y -= 15
    pdf.save()
    output.seek(0)
    return send_file(output, mimetype="application/pdf", as_attachment=True,
                     download_name=secure_filename(story.get("title","comic")) + ".pdf")

if __name__ == "__main__":
    app.run(debug=True, port=int(os.getenv("PORT", 5000)))
```

requirements.txt

```
Flask>=3.0.0
google-genai>=1.0.0
python-dotenv>=1.0.0
reportlab>=4.0.0
Pillow>=10.0.0
```

.env.example

```
GEMINI_API_KEY=PASTE_YOUR_GEMINI_API_KEY_HERE
FLASK_SECRET_KEY=replace-this-with-a-long-random-secret
GEMINI_TEXT_MODEL=gemini-3.8-flash
GEMINI_IMAGE_MODEL=gemini-3.1-flash-image
PORT=5000
```

.gitignore

```
__pycache__/
*.pyc
.env
comiccraft.db
static/generated/*.png
!static/generated/.gitkeep
.venv/
```

templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ComicCraft — AI Comic Creator</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/css2?family=Nunito:wght@400;600;700;800;900&family=Bangers&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
<header class="topbar">
  <a class="brand" href="/"><span class="brand-icon">✦</span> COMICCRAFT</a>
  <nav><a href="#creator">Create</a><a href="#library">My Comics</a></nav>
  <div class="account">
    {% if username %}
      <span class="welcome">Hi, {{ username }}</span><a class="small-btn" href="/logout">Log out</a>
    {% else %}<button class="small-btn" id="openAuth">Login / Sign up</button>{% endif %}
  </div>
</header>

<main>
<section class="hero">
  <div class="hero-copy">
    <div class="eyebrow">YOUR IMAGINATION, PANEL BY PANEL</div>
    <h1>Turn your ideas<br>into <span>COMICS!</span></h1>
    <p>Dream up a story. Let Gemini bring your characters and scenes to life. Your own comic book starts here.</p>
    <a class="primary-btn" href="#creator">Create my comic <span>■</span></a>
    <div class="hero-note">✦ Powered by Google Gemini AI</div>
  </div>
  <div class="hero-art">
    <div class="burst">WOW!</div>
    <div class="comic-card card-back">POW!</div>
    <div class="comic-card card-front"><div class="sun"></div><div class="hero-person">✦</div><b>YOUR STORY<br>STARTS HERE</b></div>
    <div class="floating f1">BAM!</div><div class="floating f2">★</div>
  </div>
</section>

<section class="creator section" id="creator">
  <div class="section-heading"><div><div class="eyebrow">THE CREATIVE STUDIO</div><h2>Build your comic</h2></div><span class="step-label">01 — S
  <div class="creator-grid">
    <div class="form-panel">
      <label for="idea">What's your story about?</label>
      <textarea id="idea" placeholder="Example: A shy young inventor discovers a tiny dragon living inside an old clock..."></textarea>
      <div class="form-row">
        <div><label for="genre">Choose a genre</label><select id="genre"><option>Adventure</option><option>Fantasy</option><option>Comedy</optio
        <div><label for="panels">Number of panels</label><select id="panels"><option value="3">3 panels</option><option value="4" selected>4 par
      </div>
      <label for="style">Illustration style</label>
      <select id="style"><option>Colorful modern comic book</option><option>Japanese manga inspired</option><option>Classic vintage comic</optio
      <button class="primary-btn full" id="generateStory">✦ Generate my story</button>
      <p class="hint">Gemini will create your title, characters, scenes and dialogue.</p>
    </div>
    <aside class="tips-panel"><div class="tips-icon">✎</div><h3>Storytelling tips</h3><ul><li>Start with one interesting character.</li><li>Giv
  </div>
</section>

<section class="results section hidden" id="results">
  <div class="section-heading"><div><div class="eyebrow">02 — YOUR AI STORY</div><h2 id="storyTitle">Your comic</h2></div><button class="outline
  <p id="storySummary" class="summary"></p>
  <div id="characters" class="characters"></div>
  <div class="panel-heading"><h3>Your comic panels</h3><button class="primary-btn" id="generateAll">✦ Illustrate all panels</button></div>
  <div id="comicPanels" class="comic-grid"></div>
  <p class="hint">Generate illustrations before saving if you want them included in your PDF.</p>
  <div class="result-actions"><button class="primary-btn" id="saveComicBottom">Save comic &amp;amp; get PDF</button></div>
</section>

<section class="library section" id="library">
  <div class="section-heading"><div><div class="eyebrow">YOUR COLLECTION</div><h2>My comic library</h2></div><button class="outline-btn" id="ref
  <div id="libraryList" class="library-list"><p class="muted">Log in to see your saved comics.</p></div>
</section>
</main>

<footer><div class="brand"><span class="brand-icon">✦</span> COMICCRAFT</div><p>Made with imagination and Google Gemini AI.</p></footer>

<div class="modal hidden" id="authModal"><div class="modal-box"><button class="close" id="closeAuth">×</button><div class="eyebrow">WELCOME TO C
  <div class="tabs"><button class="tab active" data-tab="login">Login</button><button class="tab" data-tab="register">Create account</button></d
  <form id="authForm"><div id="usernameWrap" class="hidden"><label>Username</label><input id="authUsername" autocomplete="username"></div><label
</div></div>
<div id="toast" class="toast"></div>
<script src="{{ url_for('static', filename='js/app.js') }}"></script>
</body></html>
```

static/css/style.css

```
:root{--ink:#201a35;--purple:#6c45d9;--pink:#ff668c;--yellow:#ffd65a;--cream:#fffaf0;--muted:#777287}
*{box-sizing:border-box}html{scroll-behavior:smooth}body{margin:0;background:var(--cream);color:var(--ink);font-family:Nunito,sans-serif}.topbar
@media(max-width:800px){.hero{grid-template-columns:1fr;padding:55px 7%}.hero-art{height:320px}.section{padding:55px 7%}.creator-grid{grid-templ
@media(max-width:520px){.topbar nav{display:none}.hero-art{transform:scale(.85);margin:-15px}.form-row{grid-template-columns:1fr}.section-headin
```

static/js/app.js

```
let currentStory = null;
let authMode = "login";
const $ = (id) => document.getElementById(id);

function toast(message) {
  const el = $("toast"); el.textContent = message; el.classList.add("show");
  setTimeout(() => el.classList.remove("show"), 3500);
}
async function api(url, options={}) {
  const response = await fetch(url, {headers:{"Content-Type":"application/json"}, ...options});
  const data = await response.json().catch(()=>({}));
  if (!response.ok) throw new Error(data.error || "Something went wrong.");
  return data;
}
function openAuth() { $("authModal").classList.remove("hidden"); }
function setAuthMode(mode) {
  authMode=mode;
  document.querySelectorAll(".tab").forEach(t=>t.classList.toggle("active",t.dataset.tab===mode));
  $("usernameWrap").classList.toggle("hidden",mode!=="register");
  $("authSubmit").textContent=mode==="login"?"Login":"Create account";
}
$("openAuth")?.addEventListener("click",openAuth);
$("closeAuth").addEventListener("click",()=>$("authModal").classList.add("hidden"));
document.querySelectorAll(".tab").forEach(t=>t.addEventListener("click",()=>setAuthMode(t.dataset.tab)));
$("authForm").addEventListener("submit",async e=>{
  e.preventDefault();
  const body={email:$("authEmail").value,password:$("authPassword").value};
  if(authMode==="register") body.username=$("authUsername").value;
  try{
    const result=await api(authMode==="login"?"/login":"/register",{method:"POST",body:JSON.stringify(body)});
    $("authMessage").textContent=result.message; toast(result.message); setTimeout(()=>location.reload(),700);
  }catch(err){$("authMessage").textContent=err.message;}
});

$("generateStory").addEventListener("click",async()=>{
  const idea=$("idea").value.trim();
  if(idea.length<8){toast("Please enter a little more about your story.");return;}
  const button=$("generateStory"); button.classList.add("loading");button.textContent="✦ Writing your story...";
  try{
    const result=await api("/api/generate-story",{method:"POST",body:JSON.stringify({
      idea,genre:$("genre").value,style:$("style").value,panels:$("panels").value
    })});
    currentStory=result.story; renderStory(); $("results").classList.remove("hidden");
    $("results").scrollIntoView({behavior:"smooth"}); toast("Your story is ready!");
  }catch(err){toast(err.message);}
  finally{button.classList.remove("loading");button.textContent="✦ Generate my story";}
});
function renderStory(){
  $("storyTitle").textContent=currentStory.title;
  $("storySummary").textContent=currentStory.summary||"";
  $("characters").innerHTML=(currentStory.characters||[]).map(c=>`<div class="character"><b>${escapeHtml(c.name)}</b><span>${escapeHtml(c.descri
  $("comicPanels").innerHTML=currentStory.panels.map((p,i)=>`
    <article class="comic-panel" id="panel-${i}">
      <h4>★ PANEL ${i+1}</h4>
      ${p.image_url?`<img src="${p.image_url}" alt="Comic panel ${i+1}">`:`<div class="panel-placeholder">✦ Illustration not generated yet</div>
      <p class="dialogue">“${escapeHtml(p.dialogue||"")}”</p>
      <p class="caption">${escapeHtml(p.caption||p.scene||"")}</p>
      <div class="panel-actions"><button class="mini-btn" onclick="illustratePanel(${i})">Generate image</button></div>
    </article>`).join("");
}
function escapeHtml(value){return String(value||"").replace(/[&<>"']/g,c=>({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":"&#39;"}[c]));}
async function illustratePanel(index){
  const panel=currentStory.panels[index], card=$(`panel-${index}`), btn=card.querySelector("button");
  btn.disabled=true;btn.textContent="Generating...";
  try{
    const result=await api("/api/generate-panel",{method:"POST",body:JSON.stringify({panel,story:currentStory})});
    panel.image_url=result.image_url; renderStory(); toast(`Panel ${index+1} illustrated!`);
  }catch(err){toast(err.message);btn.disabled=false;btn.textContent="Generate image";}
}
window.illustratePanel=illustratePanel;
$("generateAll").addEventListener("click",async()=>{
  for(let i=0;i<currentStory.panels.length;i++){
    if(!currentStory.panels[i].image_url) await illustratePanel(i);
  }
});
async function saveComic(){
  if(!currentStory){toast("Create a story first.");return;}
  try{
    const result=await api("/api/save-comic",{method:"POST",body:JSON.stringify({story:currentStory})});
    toast("Comic saved! Preparing PDF...");
    window.location.href=`/api/comic/${result.id}/pdf`;
    loadLibrary();
  }catch(err){toast(err.message);}
}
$("saveComic").addEventListener("click",saveComic);
$("saveComicBottom").addEventListener("click",saveComic);
async function loadLibrary(){
  try{
    const data=await api("/api/my-comics");
    $("libraryList").innerHTML=data.comics.length?data.comics.map(c=>`
      <div class="library-item"><h3>${escapeHtml(c.title)}</h3><p>${escapeHtml(c.genre)} · ${new Date(c.created_at).toLocaleDateString()}</p>
```

```
      <button class="mini-btn" onclick="openComic('${c.id}')">Open comic</button>
      <a class="mini-btn" href="/api/comic/${c.id}/pdf">PDF</a></div>`).join(""):'<p class="muted">No saved comics yet. Create your first one!</
  }catch(e){$("libraryList").innerHTML='<p class="muted">Log in to save and view comics.</p>';}
}
window.openComic=async(id)=>{
  try{const data=await api(`/api/comic/${id}`);currentStory=data.story;renderStory();$("results").classList.remove("hidden");$("results").scroll
  catch(e){toast(e.message);}
};
$("refreshLibrary").addEventListener("click",loadLibrary);
loadLibrary();
```

README.md

````
# ComicCraft — AI Comic Story Creator using Gemini Models

A full-stack Flask web application that creates comic stories and illustrated panels with Google Gemini. Users can register, generate a story, g

## Features
- User registration and login
- Gemini-powered story, character, scene and dialogue generation
- Gemini image generation for each comic panel
- Multiple genres and illustration styles
- SQLite comic library
- PDF export

## Requirements
- Python 3.10+
- A Google Gemini API key from https://aistudio.google.com/app/apikey
- Internet connection for Gemini API calls

## Setup (Windows)

1. Extract the ZIP and open the `ComicCraft` folder in VS Code.
2. Open a terminal in this folder.
3. Create a virtual environment:

   ```bash
   python -m venv venv
   venv\Scripts\activate
   ```

4. Install packages:

   ```bash
   pip install -r requirements.txt
   ```

5. Copy `.env.example` to `.env`.
6. Open `.env` and paste your Gemini API key after `GEMINI_API_KEY=`.
7. Run:

   ```bash
   python app.py
   ```

8. Open http://127.0.0.1:5000 in your browser.
9. Create an account, log in, enter an idea and generate your first comic.

## Setup (macOS / Linux)

```bash
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env and add your Gemini API key
python app.py
```

## Gemini models

Defaults in `.env.example`:
- Text: `gemini-3.8-flash`
- Image: `gemini-3.1-flash-image`

If a model is unavailable to your API key or account, select a currently available model in Google AI Studio and change the corresponding enviro

## Project structure

```text
ComicCraft/
■■■ app.py
■■■ requirements.txt
■■■ .env.example
■■■ README.md
■■■ comiccraft.db             # created automatically
■■■ templates/
■   ■■■ index.html
■■■ static/
    ■■■ css/style.css
    ■■■ js/app.js
    ■■■ generated/            # generated panel images
```

## API routes

| Method | Route | Purpose |
|---|---|---|
| GET | `/` | Main application |
| POST | `/register` | Create user |
| POST | `/login` | Log in |
| GET | `/logout` | Log out |
| POST | `/api/generate-story` | Generate story JSON |
| POST | `/api/generate-panel` | Generate one panel image |
````

POST	`/api/save-comic`	Save comic
GET	`/api/my-comics`	List saved comics
GET	`/api/comic/<id>`	Load a saved comic
GET	`/api/comic/<id>/pdf`	Download PDF

Notes and security
- Never put your Gemini API key in HTML or JavaScript. It belongs only in `.env` on the server.
- `.env` and the SQLite database are excluded from Git by `.gitignore`.
- The default Flask secret key is for local development only. Set a strong random `FLASK_SECRET_KEY` before deployment.
- This is a student-project starter, not a production authentication system. For public deployment add HTTPS, CSRF protection, rate limiting, se
- PDF export includes illustrations that have already been generated and saved with the comic.